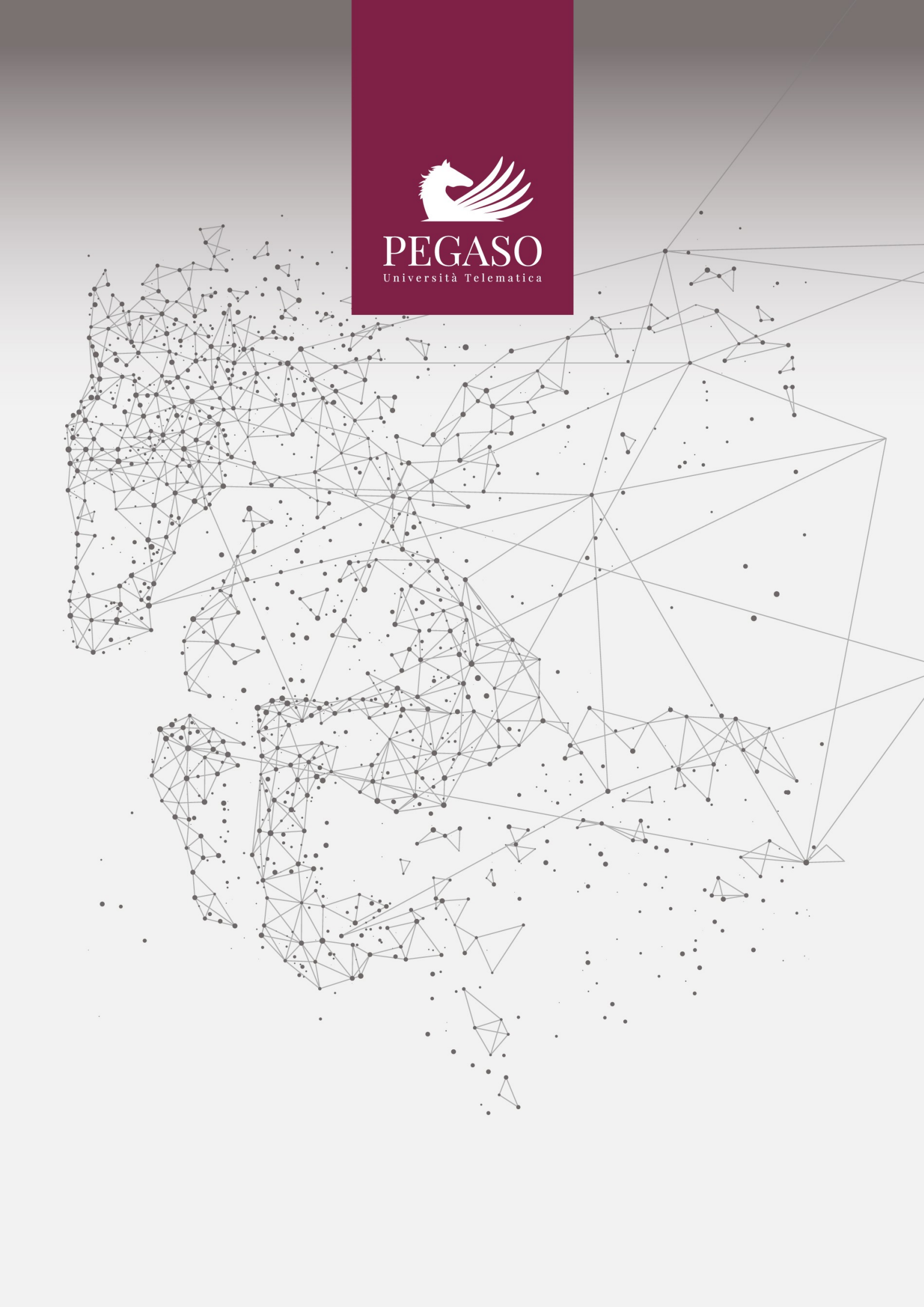

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. INTRODUZIONE A PANDAS 4
3. DATAFRAME 6
4. INDICI E POSIZIONI NEI DATAFRAME 10
5. SERIES 15
BIBLIOGRAFIA 20

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presenta Pandas, una delle librerie più importanti per la Data Science. In particolare, dopo una breve introduzione, vengono illustrate le due principali strutture dati di Pandas, i DataFrame (strutture bidimensionali che rappresentano dati tabulari) e le Series (sequenze di dati omogenei). Molto importante è anche la trattazione sviluppata in merito agli indici e alle posizioni nei DataFrame che, in alcuni casi, si utilizzano in modo diverso rispetto alle strutture dati "tradizionali" di Python.

PEGASO
Università Telematica

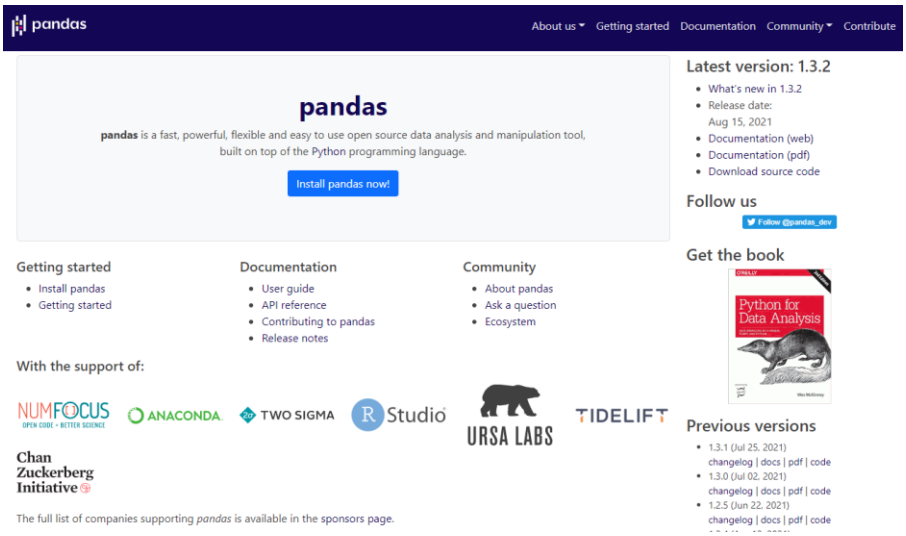
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Introduzione a Pandas

Pandas è una libreria fondata su NumPy per la manipolazione e l'analisi dei dati che:

- unifica il lavoro dell'analisi dati completando gli scenari aperti da NumPy;
- rappresenta uno dei più importanti strumenti per il data scientist (esperto di Data Science);
- permette di ridurre il tempo necessario alla preparazione dei dati;
- è caratterizzata da alte performance e facilità d'uso;
- è già presente in Anaconda.

Pandas è progettato per funzionare con tabelle o dati etichettati simili a tabelle SQL e file Excel. La pagina ufficiale di Pandas è disponibile al seguente link: https://pandas.pydata.org/



Pandas è basato su due strutture dati principali **DataFrame** e **Series** che si aggiungono al già ricco insieme di strutture dati Python. DataFrame e Series non sono semplici contenitori, ma offrono supporto per varie operazioni di manipolazioni dei dati come:

- indicizzazione monolivello e gerarchica;
- gestione dei dati mancanti;
- operazioni aritmetiche e booleane su intere colonne e tabelle;

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- operazioni tipiche da database (come l'unione e l'aggregazione);
- tracciamento grafico di singole colonne e intere tabelle;
- lettura e scrittura di dati da file.

I frame e le serie, dunque, dovrebbero essere impiegati ogni volta che si ha a che fare con **dati tabulari** (sia mono che bidimensionali).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. DataFrame

Il **DataFrame** è una struttura dati **bidimensionale** che rappresenta una sorta di **tabella di dati** da manipolare e interrogare in varie modalità, accedendovi **per righe** e **per colonne.** Per mostrare subito le potenzialità dei DataFrame, partiremo da un file CSV (Comma Separated Value) che contiene dati di tipologia diversa.

Il file CSV in questione è il file *persone.csv* (si veda Figura seguente) compilato con dati di fantasia che rappresentano determinate caratteristiche di un certo numero di soggetti. In particolare, ogni riga della «tabella» contiene dati separati da punto e virgola che sono:

1. **nome** di tipo stringa;
2. **cognome** di tipo stringa;
3. **altezza** in metri di tipo float (uso del punto come separatore dei decimali);
4. **peso** di tipo intero;
5. **data di nascita** nel formato aaaa-mm-gg;
6. **sesso** espresso con una lettera singola (M o F).

```
nome;cognome;altezza;peso;nascita;sesso
Giulio;Rossi;1.84;89;1994-08-12;M
Alessia;Verdi;1.62;53;1993-03-07;F
Enea;Bianchi;1.78;73;1996-05-06;M
Paola;Neri;1.74;61;1999-09-30;F
Ilenia;gialli;1.70;70;1998-12-27;F
Valerio;Verdi;1.88;88;1996-05-06;M
```

La funzione per la lettura automatica del CSV offerta da Pandas è **read_csv**() che prende due parametri, ovvero il **file CSV** da leggere e il **delimitatore** dei campi (nel nostro caso il punto e virgola ;).

Nel codice seguente, dunque, importiamo *pandas* e lo ridefiniamo con l'alias *pd*.

Successivamente importiamo il file *persone.csv* in un DataFrame specificando che il carattere che separa i campi gli uni dagli altri è il punto e virgola (;). Procediamo, quindi, visualizzando a schermo il tipo dell'oggetto *df*, che sarà ***pandas.core.frame.DataFrame***, e l'oggetto *df* stesso.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import pandas as pd

df = pd.read_csv('persone.csv', delimiter = ';')
print(type(df))
print()
print(df)
```

```
<class 'pandas.core.frame.DataFrame'>

      nome  cognome  altezza  peso     nascita sesso
0   Giulio    Rossi     1.84    89  1994-08-12     M
1  Alessia    Verdi     1.62    53  1993-03-07     F
2     Enea  Bianchi     1.78    73  1996-05-06     M
3    Paola     Neri     1.74    61  1999-09-30     F
4   Ilenia   gialli     1.70    70  1998-12-27     F
5  Valerio    Verdi     1.88    88  1996-05-06     M
```

Come è possibile osservare dalla visualizzazione dell'oggetto df, all'estrema sinistra di ogni riga appare un **numero progressivo** conteggiato a partire da zero. Questo aspetto rivestirà un ruolo centrale per la manipolazione dei dati.

Le colonne del DataFrame hanno quasi tutte un tipo di dato diverso. Pandas, però, ha già fatto il possibile per riconoscere i tipi di dato in modo corretto (verifica con attributo **dtypes** declinato al plurale proprio perché si riferisce alla moltitudine delle colonne). In questo caso, il tipo **object** fa riferimento ad una stringa, mentre altezza (decimale) e peso (intero) sono stati riconosciuti correttamente rispettivamente come **float64** e **int64.**

```
print(df.dtypes)
```

```
nome        object
cognome     object
altezza    float64
peso         int64
nascita     object
sesso       object
dtype: object
```

La **data**, invece, non è stata riconosciuta in automatico. Per tale motivo, si utilizza la funzione di casting di Pandas **to_datetime**() e si sfrutta la possibilità di **far riferimento ad una singola colonna** di un DataFrame con la **notazione tra parentesi quadre** *df['nascita']* come indicato nel codice seguente. L'istruzione:

df['nascita'] = pdf.to_datetime(df['nascita'])

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

permette di sostituire la colonna df['nascita'] con il risultato della sua elaborazione con la funzione to_datetime() di Pandas. A seguito del casting effettuato, si osserva che il nuovo tipo del campo 'nascita', ora è **datetime64**.

```
df['nascita'] = pd.to_datetime(df['nascita'])
print(df.dtypes)
```

```
nome                 object
cognome              object
altezza             float64
peso                  int64
nascita    datetime64[ns]
sesso                object
dtype: object
```

Prima di continuare con la trattazione dei DataFrame, è utile introdurre le funzioni **lambda**, anche note come **anonymous functions**, cioè "*funzioni senza nome*". Tali funzioni rappresentano una sintassi alternativa e più compatta rispetto alle normali funzioni da utilizzare in casi particolari.

Nel caso seguente, è possibile osservare due forme diverse per ottenere il quadrato di un numero. In particolare, a sinistra viene dichiarata la funzione quadrato() che restituisce l'argomento passato come parametro elevato al quadrato. La funzione quadrato() verrà, dunque, invocata passandogli un valore (nell'esempio 7) e il risultato sarà 49. A destra, la stessa funzionalità è ottenuta con una funzione lambda che viene contestualmente definita e invocata con il parametro passato tra parentesi (nel nostro caso (7)).

```
def quadrato(x):
    return x**2

print(quadrato(7))
```

```
49
```

```
print((lambda x : x**2) (7))
```

```
49
```

Ad ogni modo, è possibile anche denominare una funzione lambda e richiamarla nel programma come mostrato nel codice seguente.

```
quadrato = lambda x : x**2
print(quadrato(7))
```

```
49
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Le funzioni lambda sono state introdotte proprio perché potrebbero essere tra quegli elementi utili insieme alle funzioni scritte per esteso per creare nuovi campi (nuove colonne) nei DataFrame. Per poter applicare le funzioni ai DataFrame si utilizza la funzione **apply**(). A titolo di esempio, si vuole riportare, all'interno del DataFrame, anche la versione estesa del campo sesso al posto del carattere. Ne segue che riporteremo MASCHIO per M e FEMMINA per F.

Il codice (molto compatto) che ci permette di estendere il carattere è riportato di seguito.

La funzione lambda che restituisce 'MASCHIO' se x ='M' o 'FEMMINA' altrimenti, viene passata alla funzione apply() che la usa per scrivere il sesso per esteso nel nuovo campo del DataFrame denominato "sesso_esteso".

```
df['sesso_esteso'] = \
df['sesso'].apply(lambda x: 'MASCHIO' if x=='M' else 'FEMMINA')
print(df)
```

```
      nome  cognome  altezza  peso    nascita sesso sesso_esteso
0   Giulio    Rossi     1.84    89 1994-08-12     M      MASCHIO
1  Alessia    Verdi     1.62    53 1993-03-07     F      FEMMINA
2     Enea  Bianchi     1.78    73 1996-05-06     M      MASCHIO
3    Paola     Neri     1.74    61 1999-09-30     F      FEMMINA
4   Ilenia   gialli     1.70    70 1998-12-27     F      FEMMINA
5  Valerio    Verdi     1.88    88 1996-05-06     M      MASCHIO
```

Con le parentesi quadre è possibile anche **estrarre un DataFrame da un altro DataFrame**.

Per esempio, a partire dal DataFrame precedente, creiamo un DataFrame con le sole colonne nome e cognome. L'espressione *df[['nome','cognome']]* specifica che le colonne del nuovo DataFrame saranno *nome* e *cognome* e per tale motivo si passano come lista.

```
nome_cognome_df = df[['nome','cognome']]
print(type(nome_cognome_df))
print()
print(nome_cognome_df)
```

```
<class 'pandas.core.frame.DataFrame'>

      nome  cognome
0   Giulio    Rossi
1  Alessia    Verdi
2     Enea  Bianchi
3    Paola     Neri
4   Ilenia   gialli
5  Valerio    Verdi
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Indici e posizioni nei DataFrame

I DataFrame non sono gestibili solo per colonne, ma anche per righe. Questo aspetto ricorda molto da vicino l'elaborazione di file e database. I **numeri progressivi all'estrema sinistra di ogni riga del DataFrame** sono stati generati in automatico e non rappresentano la loro posizione numerica (indice), ma, sorprendentemente, sono **etichette di identificazione della riga**.

	nome	cognome	altezza	peso	nascita	sesso
0	Giulio	Rossi	1.84	89	1994-08-12	M
1	Alessia	Verdi	1.62	53	1993-03-07	F
2	Enea	Bianchi	1.78	73	1996-05-06	M
3	Paola	Neri	1.74	61	1999-09-30	F
4	Ilenia	gialli	1.70	70	1998-12-27	F
5	Valerio	Verdi	1.88	88	1996-05-06	M

Ad esempio, in Python data la seguente lista:

```
lista = ['Aldo', 'Alessio', 'Ugo', 'Elena', 'Francesca', 'Mario']
```

gli elementi saranno indicizzati in base alla loro posizione di inserimento:

- 'Aldo' in posizione 0;
- 'Alessio' in posizione 1;
- e così via.

```
print(lista[0])
print(lista[1])
```

```
Aldo
Alessio
```

Se rimuoviamo 'Aldo' (primo nome della lista), la posizione 0 sarà occupata da 'Alessio' e tutti scaleranno di un posto.

```
lista.remove('Aldo')
print(lista)
print()
print(lista[0])
```

```
['Alessio', 'Ugo', 'Elena', 'Francesca', 'Mario']

Alessio
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In una struttura dati tradizionale, gli indici rappresentano la posizione di un elemento, ma non sono un'etichetta fissa. In Pandas, non è così, infatti, esistono due operatori diversi per accedere agli elementi:

1. **loc** con cui si può accedere all'elemento *mediante la sua etichetta*;
2. **iloc** con cui si recupera il valore dell'elemento *mediante la sua posizione* all'interno del DataFrame (come per le strutture dati tradizionali).

In un DataFrame appena inizializzato, loc e iloc riferite allo stesso numero restituiranno lo stesso valore come mostrato nel codice seguente (loc a sinistra e iloc a destra).

```
print("loc[0]\n",df.loc[0])
```

```
loc[0]
 nome                    Giulio
cognome                  Rossi
altezza                   1.84
peso                        89
nascita    1994-08-12 00:00:00
sesso                        M
Name: 0, dtype: object
```

```
print("iloc[0]\n",df.iloc[0])
```

```
iloc[0]
 nome                    Giulio
cognome                  Rossi
altezza                   1.84
peso                        89
nascita    1994-08-12 00:00:00
sesso                        M
Name: 0, dtype: object
```

Dopo aver eliminato dall'elenco «Giulio Rossi» (vedremo in seguito con che funzione) che occupava la prima riga del DataFrame, loc e iloc riferite allo stesso numero non restituiranno lo stesso valore.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print("loc[0]\n",df.loc[0])
```

```
---------------------------
KeyError
C:\ProgramData\Anaconda3\lib\
   3079             try:
-> 3080                 retur
   3081             except Ke

pandas\_libs\index.pyx in par

pandas\_libs\index.pyx in par

pandas\_libs\hashtable_class_

pandas\_libs\hashtable_class_

KeyError: 0
```

```
print("iloc[0]\n",df.iloc[0])
```

```
iloc[0]
 nome                    Alessia
cognome                   Verdi
altezza                    1.62
peso                         53
nascita    1993-03-07 00:00:00
sesso                         F
Name: 1, dtype: object
```

Le differenze rispetto al caso precedente sono, infatti, le seguenti:

- *df.loc[0]* non restituisce un valore, ma solleva un'eccezione KeyError dovuto alla mancata individuazione di un elemento di «nome» 0;
- *df.iloc[0]* restituisce il primo elemento (iloc lavora sul conteggio delle posizioni e non delle etichette);
- l'output di *df.iloc[0]* all'ultima riga indica *Name:1* in quanto 1 non è la posizione del record di «Alessia Verdi», ma il suo vero e proprio nome.

Stampando l'intero DataFrame, infatti, è possibile osservare che, dopo l'eliminazione della prima riga, non troviamo nessuna che inizia con 0, ma al primo posto c'è «Alessia Verdi» con 1.

	nome	cognome	altezza	peso	nascita	sesso
1	Alessia	Verdi	1.62	53	1993-03-07	F
2	Enea	Bianchi	1.78	73	1996-05-06	M
3	Paola	Neri	1.74	61	1999-09-30	F
4	Ilenia	gialli	1.70	70	1998-12-27	F
5	Valerio	Verdi	1.88	88	1996-05-06	M

Un importante operatore che permette di mediare tra posizioni ed etichette è ***index*** che restituisce il nome di una riga (etichetta numerica) in base alla sua posizione. Considerando il DataFrame dopo aver eliminato la prima riga, il nome (etichetta) della prima riga sarà 1, come è possibile osservare dal seguente codice.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print(df.index[0])
```

```
1
```

Con loc e iloc è possibile applicare il meccanismo dello *slicing* come per le stringhe e le liste di Python. Anche in questo caso, loc lavorerà sui nomi delle righe, iloc sulle posizioni. Con *df.loc[2:4]* si richiede di recuperare tutte le righe che hanno un **nome** compreso tra 2 e 4 (estremi inclusi) come indicato di seguito.

```
print(df.loc[2:4])
```

```
     nome  cognome  altezza  peso    nascita sesso
2    Enea  Bianchi     1.78    73 1996-05-06     M
3   Paola     Neri     1.74    61 1999-09-30     F
4  Ilenia   gialli     1.70    70 1998-12-27     F
```

Con *df.iloc[2:4]* si richiede di recuperare tutte le righe che hanno una **posizione** compresa tra 2 (inclusa) e 4 (esclusa).

```
print(df.iloc[2:4])
```

```
     nome cognome  altezza  peso    nascita sesso
3   Paola    Neri     1.74    61 1999-09-30     F
4  Ilenia  gialli     1.70    70 1998-12-27     F
```

Molto interessante è la possibilità di restringere il blocco risultante dallo *slicing* anche sulle colonne dato il loro nome. All'estrazione precedente con loc vogliamo che vengano estratte solo le colonne nome e cognome. Procediamo, quindi, con il codice seguente.

```
print(df.loc[2:4][['nome','cognome']])
```

```
     nome  cognome
2    Enea  Bianchi
3   Paola     Neri
4  Ilenia   gialli
```

Se, invece, si vogliono ottenere le righe iniziali o finali de DataFrame si può operare con le seguenti funzioni:

- **head**(x) che restituisce le prime x righe del DataFrame;
- **tail**(x) che restituisce le ultime x righe del DataFrame.

  In entrambi i casi se x non indicato, il numero di righe di default è pari a 5.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print("Chiamata a head\n\n",df.head(3))
print()
print("Chiamata a tail\n\n",df.tail(2))
```

```
Chiamata a head

       nome  cognome  altezza  peso    nascita sesso
1  Alessia    Verdi     1.62    53 1993-03-07     F
2     Enea  Bianchi     1.78    73 1996-05-06     M
3    Paola     Neri     1.74    61 1999-09-30     F

Chiamata a tail

       nome cognome  altezza  peso    nascita sesso
4   Ilenia  gialli     1.70    70 1998-12-27     F
5  Valerio   Verdi     1.88    88 1996-05-06     M
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Series

Le Series sono l'altra struttura dati di Pandas oltre al DataFrame. In realtà, negli esempi precedenti, le Series sono state utilizzate senza saperlo come dimostrabile dal codice seguente in cui l'estrazione di una colonna (in questo caso quella relativa al cognome) è una Series.

```
s = df['cognome']
print(type(s))
print()
print(s)
```

```
<class 'pandas.core.series.Series'>

0      Rossi
1      Verdi
2    Bianchi
3       Neri
4     gialli
5      Verdi
Name: cognome, dtype: object
```

Il discorso è diverso se estraiamo due colonne del DataFrame (in questo caso nome e cognome); il risultato, in questo caso sarà nuovamente un DataFrame come visibile nel caso seguente.

```
s = df[['cognome','nome']]
print(type(s))
print()
print(s)
```

```
<class 'pandas.core.frame.DataFrame'>

   cognome     nome
0    Rossi   Giulio
1    Verdi  Alessia
2  Bianchi     Enea
3     Neri    Paola
4   gialli   Ilenia
5    Verdi  Valerio
```

Negli esempi precedenti, le Series sono state utilizzate senza saperlo anche con l'uso di loc e iloc, come dimostra l'esempio seguente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
sloc = df.loc[1]
siloc = df.iloc[1]
print("Tipo sloc",type(sloc))
print("Tipo siloc",type(siloc))
print()
print(sloc)
print()
print(siloc)
```

```
Tipo sloc <class 'pandas.core.series.Series'>
Tipo siloc <class 'pandas.core.series.Series'>

nome                   Alessia
cognome                  Verdi
altezza                   1.62
peso                        53
nascita    1993-03-07 00:00:00
sesso                        F
Name: 1, dtype: object

nome                   Alessia
cognome                  Verdi
altezza                   1.62
peso                        53
nascita    1993-03-07 00:00:00
sesso                        F
Name: 1, dtype: object
```

Una Series è composta da una sequenza di **elementi omogenei** ognuno individuato da un **indice**. Per questo motivo esiste la proprietà **dtype** per verificare l'unico tipo di dato della Series. Al contrario, un DataFrame contiene campi in genere di tipologie diverse.

```
h = df['altezza']

print("Series delle altezze")
```

```
Series delle altezze
0    1.84
1    1.62
2    1.78
3    1.74
4    1.70
5    1.88
Name: altezza, dtype: float64
```

```
print("Tipo elementi della Series: ",h.dtype)
```

```
Tipo elementi della Series:  float64
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Se la Series è estratta da un DataFrame, gli indici corrisponderanno a quelli del DataFrame di provenienza come è possibile verificare attraverso l'attributo **index** della Series.

```
print("Index su altezze: ",h.index)
```

```
Index su altezze:  RangeIndex(start=0, stop=6, step=1)
```

Inoltre, il risultato dell'attributo index può essere trasformato in lista con la funzione di Python **list**().

```
print("Lista indici delle altezze: ",l)
```

```
Lista indici delle altezze:  [0, 1, 2, 3, 4, 5]
```

Sempre con l'uso delle parentesi quadre possiamo recuperare i valori dalla serie applicando i principi dell'**indexing** e dello **slicing** di Python come negli esempi mostrati nel codice seguente.

```
print("Elemento in posizione esatta (zero)\n",h[0],"\n\n")
print("Tutti gli elementi fino ad una \
determinata posizione (esclusa)\n",h[:2],"\n\n")
print("Tutti gli elementi a partire da una \
determinata posizione (inclusa)\n",h[3:],"\n\n")
```

```
Elemento in posizione esatta (zero)
 1.84


Tutti gli elementi fino ad una determinata posizione (esclusa)
 0    1.84
1    1.62
Name: altezza, dtype: float64


Tutti gli elementi a partire da una determinata posizione (inclusa)
 3    1.74
4    1.70
5    1.88
Name: altezza, dtype: float64
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print("Tutti gli elementi compresi tra due posizioni\
 (prima inclusa, seconda esclusa)\n",h[2:4],"\n\n")
print("Gruppo di elementi di cui abbiamo fornito\
 sotto forma di lista, le etichette corrispondenti\n",h[[1,3,5]],"\n\n")
```

```
Tutti gli elementi compresi tra due posizioni (prima inclusa, seconda esclusa)
 2    1.78
3    1.74
Name: altezza, dtype: float64


Gruppo di elementi di cui abbiamo fornito sotto forma di lista, le etichette co
rrispondenti
 1    1.62
3    1.74
5    1.88
Name: altezza, dtype: float64
```

Una Series può essere prodotta anche partendo da strutture dati Python. Il primo esempio è la creazione di una serie a partire da una **lista** come mostrato nel codice seguente.

```
import pandas as pd

capitali = pd.Series(data = ['Roma','Parigi','Madrid','Lisbona','Berlino'])
print(capitali)
```

```
0       Roma
1     Parigi
2     Madrid
3    Lisbona
4    Berlino
dtype: object
```

Il secondo esempio è la creazione di una serie a partire da un **dizionario** come mostrato nel codice seguente.

```
import pandas as pd

dizionario_capitali = {'Italia':'Roma','Francia':'Parigi',\
                       'Spagna':'Madrid','Portogallo':'Lisbona',\
                       'Germania':'Berlino'}
capitali_da_dizionario = pd.Series(dizionario_capitali)
print(capitali_da_dizionario)
```

```
Italia          Roma
Francia       Parigi
Spagna        Madrid
Portogallo   Lisbona
Germania     Berlino
dtype: object
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il terzo esempio è la creazione di una serie a partire da un **array NumPy** come mostrato nel codice seguente.

```
import pandas as pd
import numpy as np

c = np.array(['Roma','Parigi','Madrid','Lisbona','Berlino'])
capitali = pd.Series(c)
print(capitali)
```

```
0       Roma
1     Parigi
2     Madrid
3    Lisbona
4    Berlino
dtype: object
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev D. (2017). Data science con Python. Apogeo.
- Maggi G. (2020). Data science con Python. Edizioni LSWR.
- Marin I., Shukla A., Sarang VK. (2019). L'analisi dei Big Data con Python. Tecniche nuove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).